RISC-V 시뮬레이터 구현 프로젝트

본 프로젝트에서는 지정된 RISC-V 명령어를 실행하는 기능 시뮬레이터를 구현한다. 프로그래밍 언어는 제한하지 않으며, 정확한 명령어 실행과 바이너리 입출력, 테스트 및 개인별 이해도를 평가한다. 파이프라인·캐시·사이클 단위 타이밍 모델은 필수 구현 범위에 포함하지 않는다.

1. 수행 방식과 학습목표

팀은 3~5 명으로 구성한다. 언어·개발 도구는 자유롭게 선택하되, 제출 환경에서 소스 코드로 빌드 또는 실행할 수 있어야 한다. 모든 팀원은 담당 부분뿐 아니라 공통 실행 원리와 최종 제출물의 동작을 이해해야 한다.

학습목표: ① 명령어 인코딩과 해독 ② 32 비트 레지스터와 부호 해석 ③ 메모리 주소·바이트 순서 ④ 조건 분기와 PC 갱신 ⑤ 기대 결과에 근거한 검증 ⑥ 자신의 구현에 대한 설명과 오류 진단.

2. 실행 모델과 주소 공간

항목	요구사항
명령어 모델	RV32I 중 본 문서에 열거된 14 개 명령어만 지원
레지스터	32 개의 32 비트 정수 레지스터 x0~x31. x0 는 항상 0
초기 PC	0x00040000
text segment 시작 주소	0x00040000
data segment 시작 주소	0x10000000
바이트 순서	명령어·데이터 모두 little-endian
종료	지원하지 않는 32 비트 명령어가 fetch 되면 실행하지 않고 종료
필수 파일 출력	data.out: 데이터 시작 주소부터 정확히 1,024 바이트
필수 표준 출력	종료 명령어 주소와 값을 지정된 형식으로 한 줄 출력

본 시뮬레이터는 ISA 수준의 상태 변화를 구현한다. 호스트 언어의 정수 크기·부호 규칙·파일 직렬화 방식이 시뮬레이터의 의미를 바꾸어서는 안 된다. 덧셈·뺄셈 결과와 레지스터에 저장되는 값은 하위 32 비트를 유지한다.

3. 지원 명령어

아래 표의 RISC-V 명령어들을 지원한다.

명령어	형식
add	R
sub	R
addi	I
and / or	R
andi / ori	I
lui	U
lw	I
sw	S
beq / bne	B
blt / bge	B

4. 프로그램 입력

입력은 헤더 없는 바이너리 파일 text.bin 과 data.bin 이다. text.bin 의 첫 바이트는 0x00040000, data.bin 의 첫 바이트는 0x10000000 에 배치한다. 이후 바이트는 파일 순서대로 연속된 주소에 적재한다. ASCII 문자열이나 16 진수 텍스트 파일이 아니다.

주소 a 부터 네 바이트가 b0, b1, b2, b3 이면 32 비트 값은 b0 + (b1<<8) + (b2<<16) + (b3<<24)이다. 예를 들어 바이트 78 56 34 12 는 값 0x12345678 을 나타낸다.

5. 종료와 출력

5.1 종료 순서와 표준 출력

invalid instruction 을 fetch 하면 이를 실행하지 않고 정상적인 종료 처리로 이동한다. 종료 시 출력하는 PC 는 해당 명령어를 fetch 한 주소이며 PC+4 가 아니다. 구현 내부에서 PC 를 미리 증가시키는 경우에도 fetch 당시 주소를 보존해야 한다.

```
PC 0x00040010 invalid instruction 0x00000000
```

출력 규칙: "PC"는 대문자, 나머지 고정 문자열은 예시와 같다. 각 항목 사이에는 공백 한 칸을 사용한다. 두 숫자는 모두 0x 접두사와 선행 0 을 포함한 소문자 8 자리 16 진수로 출력하고, 마지막에 줄바꿈을 붙인다. 표준 출력에는 추가 안내문·디버깅 기록을 출력하지 않는다. 필요한 진단 정보는 표준 오류 또는 별도 파일을 사용한다.

명령어 값은 입력 바이트 네 개를 little-endian 으로 해석한 32 비트 값이다. 바이트를 파일 순서대로 나열한 문자열과 혼동하지 않는다.

5.2 data.out

종료 처리에서 메모리 주소 0x10000000 부터 0x100003FF 까지를 주소 증가 순서로 data.out 에 저장한다. 파일 크기는 정확히 1,024 바이트이며 헤더·주소 문자열·공백·줄바꿈을 넣지 않는다. 기존 data.out 이 있으면 새 결과로 덮어쓴다. invalid instruction 뒤의 명령어는 실행하지 않으므로 그 명령어의 효과가 출력에 반영되어서는 안 된다.

현재 명세의 필수 출력은 표준 출력 한 줄과 data.out 이다. 레지스터 덤프 파일은 필수 제출 출력이 아니며 디버깅 목적으로 별도 생성할 수 있다.

6. 초기 상태와 유효 입력 범위

항목	확정 조건
초기 레지스터	x0~x31 모두 0. 별도의 스택 포인터 초기화 없음
데이터 메모리	0x10000000~0x100003FF 의 1,024 바이트만 필수 지원
data.bin 크기	0~1,024 바이트. 적재 전 전체 데이터 메모리를 0 으로 초기화
text.bin 크기	4 바이트의 양의 정수배. 마지막 4 바이트에 0x00000000 포함
정상 종료 경로	모든 채점 프로그램은 실행 중 invalid instruction 에 도달하도록 구성
명령어 정렬	실행되는 모든 명령어 주소는 4 바이트 정렬
데이터 정렬	lw·sw 유효 주소는 4 바이트 정렬이며 4 바이트 전체가 데이터 영역 안에 위치
메모리 분리	명령어는 적재된 text segment 에서 fetch, lw·sw 는 data 영역 접근. text segment 쓰기 불가능
지원 제외	압축 명령어, 시스템 호출, 예외·인터럽트, 자기 수정 코드, 운영체제 기능
실행 제한	정상 명령어를 최대 1,000,000 개 실행한다. 이후 다음 fetch 가 invalid 이면 정상 종료하고, 지원 명령어이면 실행하지 않고 실행 한도 초과로 비정상 종료

정상 기능 채점 입력은 파일 크기·정렬·메모리 범위 조건을 만족하며 실행 한도 안에 invalid instruction 에 도달한다.

시뮬레이터 실행 시 입력 경로 두 개를 인수로 받는다. 예: <실행명령> text.bin data.bin. 출력 data.out 은 현재 작업 디렉터리에 생성한다. 언어별 실제 실행·빌드 명령은 README 에 명시한다. argv 의 실행명 자체를 제외한 인수는 text 파일 경로, data 파일 경로 순서의 두 개이다. 실행 중 네트워크·AI 서비스·대화형 입력에 의존해서는 안 된다.

7. 기능 확인 예시

다음은 입출력 의미를 설명하는 예시이며, 전체 테스트를 대체하지 않는다. data.bin 은 1,024 바이트의 0 으로 구성했다고 가정한다.

```
주소         기계어       의미
0x00040000  0x100000b7   lui  x1, 0x10000
0x00040004  0x00700113   addi x2, x0, 7
0x00040008  0x00500193   addi x3, x0, 5
0x0004000c  0x00310233   add  x4, x2, x3
0x00040010  0x0040a023   sw   x4, 0(x1)
0x00040014  0x00000000   invalid instruction
```

위 프로그램의 text.bin 바이트열은 다음과 같다. 아래는 설명용 16 진수 표기이며, 실제 파일에는 해당 바이트를 저장한다.

```
b7 00 00 10 13 01 70 00 93 01 50 00
33 02 31 00 23 a0 40 00 00 00 00 00
```

기대 표준 출력:

```
PC 0x00040014 invalid instruction 0x00000000
```

data.out 의 첫 4 바이트는 0c 00 00 00 이며 나머지 1,020 바이트는 0 이다. 파일 전체 크기는 1,024 바이트이다.

8. 제출물과 보고서

팀별로 소스 코드와 보고서를 제출한다. 아래 파일 구성을 반드시 포함한다. 보고서는 pdf file 로 제출한다.

제출물	포함 내용
소스 코드	실행에 필요한 전체 소스와 자체 작성 보조 파일
README	팀원, 언어·버전, 의존성, 빌드·실행 명령, 파일 배치
보고서(.pdf)	설계, 구현 위치, 검증, 제한사항, 역할, AI 활용
자체 테스트 입출력	text.bin·data.bin, 기대 출력, 테스트 목적 또는 생성 코드
사진	프로젝트 실행 중 팀원 전원이 모여 함께 찍은 사진

보고서 필수 구성: ① 지원 범위와 실행 방법 ② PC·레지스터·메모리 표현 ③ fetch/decode/execute 및 PC 갱신 흐름 ④ 즉시값·부호·32 비트 처리 ⑤ 바이너리 입출력 ⑥ invalid 판별과 종료 처리 ⑦ 테스트와 기대 결과의 도출 ⑧ 발견한 오류와 수정 근거 ⑨ 알려진 제한사항 ⑩ 팀원별 기여 ⑪ AI 도구 정보·활용 정도 및 검증.

기능별 구현 위치표를 포함한다. 최소 항목은 명령어 인출, R/I/S/B/U 형 처리, lw·sw, 분기, invalid 판별, data.out 생성이며 각 항목에 파일명·함수명과 짧은 설명을 기재한다. 제출 마감 후 소스와 보고서의 버전을 고정하며 개인 평가는 이 제출본을 기준으로 한다.

언어 제한은 없지만, 일반 프로그래밍 라이브러리를 이용하는 것과 완성된 RISC-V 실행기를 호출하는 것은 구분한다. 파일 입출력·컨테이너 등 일반 라이브러리는 허용한다. 완성된 RISC-V 시뮬레이터·에뮬레이터를 실행 엔진으로 사용하거나 그 핵심 실행기를 그대로 제출하는 것은 금지한다. 참조 실행기를 별도 테스트용으로 사용하는 것은 허용하되 보고서에 밝힌다. 참고 코드·자료·외부 의존성과 AI 생성 코드의 활용은 출처와 범위를 밝힌다.

9. AI 활용과 개인 이해도 평가

프로젝트 수행 중 AI 활용은 허용한다. 다만 코드·보고서·테스트·결과의 정확성은 제출자에게 책임이 있다. AI 가 생성했다는 이유로 오류의 책임이 면제되지 않는다. 사용한 AI 도구 정보와 활용 정도를 이 절의 양식에 따라 반드시 보고서에 기록한다. 중요한 활용 사례에는 목적, 제안 내용, 검증 방법, 채택·수정·기각 이유를 포함한다. AI 를 사용하지 않은 경우 그 사실과 자체 검증 과정을 기록한다. 전체 대화 기록 제출은 기본 요구가 아니다.

기말고사 시간에 프로젝트 관련 문항에 AI 없이 개별 답안을 작성한다. 이 문항의 점수는 기말고사 성적이 아니라 프로젝트의 개인 평가 점수에 반영한다. 같은 공통 질문이라도 팀의 구현에 따라 답이 달라질 수 있으며, 같은 팀 구성원도 독립적으로 답안을 작성한다.

개인 문항은 자신의 코드·보고서 설명, 실행 결과 예측, 오류 진단, 테스트 설계 및 작은 변경의 영향 분석을 포함할 수 있다. 담당 부분뿐 아니라 필수 기능의 공통 원리를 이해해야 한다.

시험에서는 AI·인터넷·전자기기·상호 협의를 금지한다. 제출 마감 시 고정된 본인 팀의 소스 코드를 인쇄한 자료를 열람할 수 있다. 인쇄본의 내용은 제출본과 일치해야 하며, 별도 해설·예상 답안·추가 메모는 허용하지 않는다. 시험에는 새로운 입력·검증 상황을 포함한다.

9.1 개인 평가 문항 예시

- 본인 팀의 little-endian 입력 처리를 설명하고, 바이트 78 56 34 12 로 구성되는 값을 도출하시오.
- 본인 팀의 blt 구현 위치를 제시하고 0xffffffff 와 1 을 비교할 때 분기 여부를 설명하시오.
- 본인 팀에서 invalid instruction 을 판별하는 과정을 설명하고, fetch 당시 PC 를 출력하는 근거를 제시하시오.

프로젝트 결과와 개인 답안의 차이는 이해도 확인의 근거이며, 그 자체가 AI 부정사용의 확정 증거는 아니다. 필요 시 동일한 기준의 추가 확인을 실시할 수 있다.

9.2 보고서 필수 항목: AI 도구 정보와 활용 정도

팀 전체 사용 내역을 정리하고 사용자(팀원)를 표시한다. 여러 도구·모델을 사용한 경우 각각 기록한다. 모델·버전이 표시되지 않는 서비스는 "확인 불가"라고 쓰며 추정하여 기재하지 않는다. 계정 정보·비밀번호·API 키 등 비밀정보는 제출하지 않는다.

필수 항목	작성 내용
도구 식별	서비스·제품명, 제공사, 모델명·버전(확인 가능한 범위), 사용 시기
사용자·사용 방식	사용 팀원, 웹 대화형·IDE 자동완성·에이전트 등 사용 방식
목적·대상	개념 학습, 설계, 명령어 구현, 입출력, 테스트, 디버깅, 보고서 등
활용 정도	영역별 등급 0~3 과 그 근거. 기능명·파일명·함수명을 함께 기록
채택·수정 범위	어떤 제안을 어디에 반영했고 얼마나 수정했는지 구체적으로 설명
검증	독립 기대 결과, 자체 테스트, 참조 결과 등 검증 방법과 결과
대표 사례	핵심 요청·제안 요약, 채택·수정·기각 이유와 오류 수정 사례

아래 등급은 사용량을 비교하거나 감점하기 위한 것이 아니라, AI 가 수행 과정에서 맡은 역할을 일관되게 설명하기 위한 자기보고 기준이다. 여러 등급이 섞인 영역은 가장 높은 등급을 표시하고 하위 범위를 설명한다.

등급	활용 정도의 정의
0: 미사용	해당 영역에서 AI 를 사용하지 않음
1: 보조	개념 설명·아이디어·문장 수정 등을 받았으나 핵심 코드·분석은 직접 작성
2: 부분 생성	일부 함수·테스트·분석 초안을 생성받아 검토·수정하여 반영
3: 주요 생성	주요 모듈 또는 해당 영역의 대부분을 생성받고 통합·수정·검증

영역별 활용표는 다음 항목을 모두 작성한다: 전체 설계 / 명령어 해독·즉시값 / 산술·논리 연산 / 메모리·바이너리 입출력 / 분기·종료 / 테스트·디버깅 / 보고서. 각 행에 사용 도구, 사용자, 등급, 반영 위치, 검증 근거를 제시한다. 등급은 자동 측정값이 아니라 학생의 근거 있는 자기보고이다.

기록 누락·설명 부족은 보고서 완성도 기준으로 평가하며, 확인된 허위 기재는 대학의 학업윤리 절차에 따라 처리한다.

10. 평가 기준과 배점

프로젝트는 100 점 만점이며 개인별 프로젝트 점수 = 팀 결과물 점수(60 점) + 개인 이해도 점수(40 점)로 산정한다. 기말고사 시간에 실시하는 프로젝트 문항의 점수는 프로젝트의 개인 이해도 점수에만 반영하고, 기말고사 자체 점수에 중복 반영하지 않는다.

기능 평가는 공통 입력에 대한 종료 출력과 data.out 의 바이트 단위 비교를 중심으로 수행한다. 공개 테스트 외 비공개 테스트를 사용할 수 있다. 고급 언어 기능, GUI, 코드 분량, 개발 언어, 시뮬레이터 자체의 속도, 필수 범위를 넘는 기능 수는 기본 가산점 항목으로 삼지 않는다. 팀 인원에 관계없이 동일한 필수 기능과 기준을 적용한다.

평가자는 AI 를 제출물 검토·문항·피드백 초안 작성의 보조 수단으로 사용할 수 있다. 정답은 독립적으로 검증하며 최종 점수와 이의 검토는 담당교원이 결정한다. 학생의 AI 사용량 자체를 가감점 사유로 삼지 않고 결과의 정확성과 개인 이해도를 평가한다.

11. 제출 전 체크리스트

☐ 지원 14 개 명령어와 x0 고정 동작을 확인했다.

☐ text 시작 주소 0x00040000 과 data 시작 주소 0x10000000 을 적용했다.

☐ little-endian 입출력과 lw·sw 를 확인했다.

☐ 음수 즉시값, blt·bge 부호 비교, 분기 PC 계산을 검증했다.

☐ opcode 뿐 아니라 필요한 funct3·funct7 까지 판별했다.

☐ invalid instruction 은 실행하지 않으며 fetch 당시 PC 를 출력한다.

☐ 표준 출력 한 줄의 공백·8 자리 소문자 16 진수·줄바꿈을 확인했다.

☐ data.out 은 시작 주소부터 정확히 1,024 바이트이며 바이너리 화일이다.

☐ 새 환경에서 README 만으로 빌드·실행할 수 있는지 확인했다.

☐ 보고서의 구현 위치·설명과 최종 소스가 일치한다.

☐ AI 도구 정보와 영역별 활용 등급·반영 위치·검증 근거를 기록했다.

☐ 모든 팀원이 자신의 제출물을 설명하고 테스트 결과를 도출할 수 있다.